

Selected Solutions for Chapters 1–6

Source: docs/guidebook.pdf, chapters 1 through 6.

Selection criterion: I chose two exercises from each chapter that expose core computer-architecture ideas rather than only lookup facts. Together they cover pipeline timing, fixed-width integer behavior, assembler data layout, ARM arithmetic translation, load/store addressing, and conditional control flow.

Chapter 1, Exercise 12: ideal three-stage pipeline

For an ideal pipeline with three stages, no stalls, no branch penalties, no data hazards, and no memory wait states, the pipeline must first be filled. After that, one instruction completes on every clock cycle.

For n instructions and $k = 3$ stages:

$$\text{cycles}(n) = k + (n - 1) = n + 2.$$

For 10 instructions:

$$\text{cycles}(10) = 10 + 2 = 12 \text{ cycles}, \quad \text{throughput} = \frac{10}{12} = \frac{5}{6}.$$

For n instructions:

$$\text{throughput}(n) = \frac{n}{n + 2}.$$

As n becomes large, the fixed two-cycle fill cost becomes negligible:

$$\lim_{n \rightarrow \infty} \text{throughput}(n) = 1 \text{ instruction per cycle}.$$

The important distinction is latency versus throughput. A single instruction still needs three stages to pass through the machine, but once the pipeline is full, completions occur at the rate of one instruction per cycle under the ideal assumptions.

Chapter 1, Exercise 13: non-pipelined versus pipelined speed

Processor A is non-pipelined at 10 MHz. Processor B is an ideal three-stage pipeline at 28 MHz. To compare execution time for the same instruction stream, assume both processors perform the same three logical stages per instruction and that Processor B has the ideal no-stall behavior from Exercise 12.

For n instructions:

$$\begin{aligned} \text{cycles}_A(n) &= 3n, & \text{time}_A(n) &= \frac{3n}{10 \text{ MHz}}, \\ \text{cycles}_B(n) &= n + 2, & \text{time}_B(n) &= \frac{n + 2}{28 \text{ MHz}}. \end{aligned}$$

The speedup of B over A is:

$$\text{speedup}(n) = \frac{\text{time}_A(n)}{\text{time}_B(n)} = \frac{3n/10}{(n+2)/28} = \frac{8.4n}{n+2}.$$

For a sufficiently long program:

$$\lim_{n \rightarrow \infty} \text{speedup}(n) = 8.4.$$

Thus Processor B is approximately 8.4 times faster for long straight-line code. For short programs the speedup is smaller because the pipeline fill cost is not negligible. For $n = 10$, the speedup is:

$$\text{speedup}(10) = \frac{8.4 \cdot 10}{12} = 7.0.$$

The result shows that the higher clock frequency and the pipeline both matter: the frequency ratio is only $28/10 = 2.8$, while the ideal pipeline also improves instruction throughput by roughly a factor of three after the pipeline is full.

Chapter 2, Exercise 2: six-bit two's-complement arithmetic

In a six-bit two's-complement system, the representable signed range is:

$$-32 \leq x \leq 31.$$

The carry flag C is interpreted as the unsigned carry-out for addition. For subtraction, $C = 1$ means no borrow and $C = 0$ means borrow. The signed overflow flag V is set when the mathematical signed result is outside the representable range. For multiplication, a conventional add/subtract carry flag is not meaningful; ARM multiply instructions do not define C or V. I therefore report multiplication overflow by checking whether the exact product fits in six signed bits.

Operation	Six-bit operation	Stored	Signed	C	V
-7 + (-29)	111001 + 100011 = 1 011100	011100	28	1	1
31 + 11	011111 + 001011 = 0 101010	101010	-22	0	1
15 - 19	001111 + 101101 = 0 111100	111100	-4	0	0
-7 * (-3)	exact product 21	010101	21	N/A	0
-7 * 3	exact product -21	101011	-21	N/A	0
21 + 3	010101 + 000011 = 0 011000	011000	24	0	0
21 + (-3)	010101 + 111101 = 1 010010	010010	18	1	0

Two examples show why C and V are different facts. In $-7 + (-29)$, the unsigned addition has a carry-out and the signed result overflows because -36 is below -32 . In $21 + (-3)$, there is a carry-out in the unsigned addition, but the signed result 18 is representable, so signed overflow is clear.

Chapter 2, Exercise 3: why signed GE tests $N == V$

For a signed comparison, the processor conceptually evaluates $A - B$ and updates the condition flags. The signed greater-or-equal condition means $A \geq B$, which is equivalent to saying that the true mathematical result $A - B$ is non-negative.

If there is no signed overflow, $V = 0$, and the negative flag N correctly reports the sign of the subtraction result:

$$N = 0 \Rightarrow A - B \geq 0 \Rightarrow A \geq B, \quad N = 1 \Rightarrow A - B < 0 \Rightarrow A < B.$$

If signed overflow occurs, $V = 1$, and the sign bit stored in the fixed-width result is inverted relative to the true mathematical sign. In that case, $N = 1$ actually corresponds to a non-negative mathematical result, and $N = 0$ corresponds to a negative mathematical result.

Therefore the corrected sign of $A - B$ is captured by:

$$N \oplus V.$$

The result is signed non-negative exactly when this corrected sign is zero:

$$N \oplus V = 0,$$

which is equivalent to:

$$N = V.$$

That is why ARM uses $N = V$ for the signed greater-or-equal condition.

Chapter 3, Exercise 7: `ALIGN 8, 5` and data layout

In the assembler syntax used by the text, `ALIGN boundary, offset` advances the current location counter until the address is congruent to `offset` modulo `boundary`. Thus `ALIGN 8, 5` means: skip bytes until the next address whose low address bits place it at offset 5 within an 8-byte block. Equivalently, find the next address A such that:

$$A \bmod 8 = 5.$$

Given:

```
AREA myData, Data
a   ALIGN 4
b   DCB 1
c   DCB 2
    DCB 3
d   ALIGN 8,5
    DCB 5
```

Assume the data area starts at `0x20000000`. `ALIGN 4` at the beginning adds no padding because `0x20000000 mod 4 = 0`. Then the three `DCB` directives place one byte each.

Address	Content	Meaning
0x20000000	0x01	label b
0x20000001	0x02	label c
0x20000002	0x03	unnamed byte
0x20000003	0x00	padding inserted by ALIGN 8,5
0x20000004	0x00	padding inserted by ALIGN 8,5
0x20000005	0x05	byte after label d alignment

The next address congruent to 5 modulo 8 after 0x20000003 is 0x20000005, so two padding bytes are required.

Chapter 3, Exercise 8: errors in the assembly program

The given program is:

```

AREA myData, DATA, READWRITE
String DCB "ABCDE"
Array DCD 1, 2, 3, 4, 5
END
AREA myCode, CODE, READONLY
EXPORT _main2
main PROC

sum

PROC
ENDP
ENDP
END

```

The main problems are structural and symbolic:

1. **END** appears immediately after the data area. **END** terminates the assembly source, so the following code area would be ignored or treated as invalid trailing text. There should be only one **END**, at the end of the source file.
2. The exported symbol is **_main2**, but no **_main2** label or procedure is defined. The procedure label is **main**, so the exported name and the actual entry symbol do not match.
3. If this source is meant to be the program entry used by the book's startup path, the entry procedure should normally be named and exported consistently as **_main** or whatever the startup code calls. Exporting **_main2** changes the public contract without providing that symbol.
4. **main PROC** is not closed before defining another procedure. Procedures should not be nested this way; **main** needs its own **ENDP** before **sum PROC** begins.
5. The **sum** procedure is split across lines as a bare label followed by **PROC**. It should be written consistently as **sum PROC**.

6. `String DCB "ABCDE"` allocates five bytes but no null terminator. That is legal for a raw byte array, but it is incorrect if the intent is a C-style string. For a null-terminated string, use `String DCB "ABCDE", 0`.

A corrected skeleton is:

```

        AREA myData, DATA, READWRITE
String  DCB "ABCDE", 0
Array   DCD 1, 2, 3, 4, 5

        AREA myCode, CODE, READONLY
        EXPORT _main
        ENTRY

_main    PROC
        ; program body
        B .
        ENDP

sum      PROC
        ; subroutine body
        BX lr
        ENDP

        END

```

The important rule is that assembler symbols are contracts. The startup code, exported names, labels, and procedure boundaries must agree.

Chapter 4, Exercise 6: polynomial evaluation in ARM assembly

The target expression is:

$$y = 3x^3 - 7x^2 + 10x - 11.$$

Assumption: signed integer x is in `r0`, the result y is returned in `r1`, and the mathematical result fits in 32 signed bits. The most compact structure is Horner's rule:

$$y = ((3x - 7)x + 10)x - 11.$$

One ARM/Thumb-2 implementation is:

```

; input:  r0 = x
; output: r1 = 3*x^3 - 7*x^2 + 10*x - 11
; clobber: r1

ADD     r1, r0, r0, LSL #1    ; r1 = 3*x
SUB     r1, r1, #7            ; r1 = 3*x - 7
MUL     r1, r1, r0            ; r1 = (3*x - 7)*x

```

```

ADD    r1, r1, #10          ; r1 = (3*x - 7)*x + 10
MUL    r1, r1, r0          ; r1 = ((3*x - 7)*x + 10)*x
SUB    r1, r1, #11         ; r1 = final y

```

This form avoids explicitly materializing x^2 and x^3 . It also keeps the live register set small, which is valuable in assembly because register pressure directly affects whether stack traffic is needed.

Chapter 4, Exercise 10: reverse byte order without REV

The goal is to transform $r0 = [B3\ B2\ B1\ B0]$ into $r1 = [B0\ B1\ B2\ B3]$ without using the REV instruction. The following implementation uses masks, shifts, and OR operations. It assumes $r0$ holds the input word, returns the reversed word in $r1$, and uses $r2$ as a temporary register.

```

; input:  r0 = 0xB3B2B1B0
; output: r1 = 0xB0B1B2B3
; clobber: r2

AND     r1, r0, #0x000000FF
LSL     r1, r1, #24          ; move B0 to bits [31:24]

AND     r2, r0, #0x0000FF00
LSL     r2, r2, #8           ; move B1 to bits [23:16]
ORR     r1, r1, r2

AND     r2, r0, #0x00FF0000
LSR     r2, r2, #8           ; move B2 to bits [15:8]
ORR     r1, r1, r2

LSR     r2, r0, #24          ; move B3 to bits [7:0]
ORR     r1, r1, r2

```

The operation is purely combinational at the bit level: each byte is isolated, moved to its new byte lane, and merged into the result. This is exactly what REV does in one instruction, but the expanded version makes the data movement explicit.

Chapter 5, Exercise 6: address used for r7 in multiple loads

The register list is sorted by register number before the transfer. For all four instructions, the logical order is:

$r1, r2, r3, r6, r7.$

The base register starts as $r0 = 0x20008000$. The instructions in the exercise do not include `!`, so write-back is disabled. Therefore $r0$ remains $0x20008000$ after each instruction.

Instruction	Address used for r7	Final r0
LDMIA r0, {r1, r3, r7, r6, r2}	0x20008010	0x20008000
LDMIB r0, {r1, r3, r7, r6, r2}	0x20008014	0x20008000
LDMDA r0, {r1, r3, r7, r6, r2}	0x20008000	0x20008000
LDMDB r0, {r1, r3, r7, r6, r2}	0x20007FFC	0x20008000

Increment-after starts at r0, so the fifth register r7 is loaded from r0 + 16. Increment-before starts at r0 + 4, so r7 is loaded from r0 + 20. Decrement-after uses a descending block ending at r0; the highest-numbered register in this five-register list, r7, is loaded from r0. Decrement-before uses a descending block ending at r0 - 4; therefore r7 is loaded from r0 - 4.

Chapter 5, Exercise 9: PC-relative address range

The instruction address is 0x10004000. For Cortex-M Thumb execution, the PC value used by many PC-relative calculations is the address of the current instruction plus 4. Therefore the effective PC base is:

$$PC_{base} = 0x10004000 + 4 = 0x10004004.$$

The exercise states that the PC-relative offset is a 12-bit signed integer. A signed 12-bit integer ranges from -2048 through +2047. Therefore the byte address range is:

$$\text{minimum} = 0x10004004 - 0x800 = 0x10003804,$$

$$\text{maximum} = 0x10004004 + 0x7FF = 0x10004803.$$

So the target data can be located in:

$$0x10003804 \text{ through } 0x10004803.$$

If the target is a word loaded by LDR, a practical aligned literal should be word-aligned. Under that alignment constraint, the highest aligned address in the range is 0x10004800. Thus the word-aligned practical range is:

$$0x10003804 \text{ through } 0x10004800.$$

Chapter 6, Exercise 14: factorial in ARM assembly

The target function is:

$$f(n) = n! = n(n-1) \cdots 2 \cdot 1.$$

Assumptions: n is a non-negative integer in r0; the result is returned in r1; r0 is preserved by copying it into r2; and the result fits in 32 bits. Without that assumption, overflow detection or a wider result convention would be required.

```
; input:  r0 = n
; output: r1 = n!
```

```

; clobber: r2

MOV    r1, #1          ; 0! and 1! both produce 1
MOV    r2, r0          ; preserve the input n in r0

factorial_loop
    CMP    r2, #1
    BLS    factorial_done ; if r2 <= 1, the product is complete
    MUL    r1, r1, r2
    SUBS    r2, r2, #1
    B      factorial_loop

factorial_done
    ; r1 contains n!

```

The loop invariant is that **r1** contains the product of all original factors already consumed, and **r2** contains the next factor to multiply unless **r2** <= 1. Initialization gives **r1** = 1, which is the multiplicative identity and also the correct result for 0! and 1!. Each iteration multiplies by the current factor and decrements it. When **r2** <= 1, all factors greater than one have been consumed, so **r1** is the factorial.

Chapter 6, Exercise 8: test for complex roots

For the quadratic equation:

$$ax^2 + bx + c = 0,$$

the roots are complex exactly when the discriminant is negative:

$$D = b^2 - 4ac < 0.$$

Assume signed integers **a**, **b**, and **c** are in **r0**, **r1**, and **r2**, respectively. The result is stored in **r3**, where **r3** = 1 if roots are complex and **r3** = 0 otherwise.

Assume the intermediate values **b*b**, **a*c**, **4*a*c**, and **D** fit in signed 32-bit range. If that cannot be assumed, the safe version should use **SMULL** and a 64-bit subtract.

```

; input:  r0 = a, r1 = b, r2 = c
; output: r3 = 1 if b*b - 4*a*c < 0, else 0
; clobber: r4, r5

MUL    r4, r1, r1      ; r4 = b*b
MUL    r5, r0, r2      ; r5 = a*c
LSL    r5, r5, #2      ; r5 = 4*a*c
SUBS    r4, r4, r5      ; r4 = discriminant, flags updated

MOV    r3, #0          ; assume real roots
BPL    roots_done      ; D >= 0 means real roots
MOV    r3, #1          ; D < 0 means complex roots

roots_done
    ; r3 contains the answer

```


The key branch is BPL, which means "branch if plus", i.e. branch when the negative flag is clear. After SUBS, the negative flag reflects the sign of the discriminant under the stated no-overflow assumption.